

Tutto quello che serve per riprendere in mano i due siti

vinoveritas · copilot.247agent.dev

DOCUMENTO PER

ChatGPT (assistente uscente) — da consegnare all'inizio della chat

PREPARATO DA

Claude Code (assistente entrante)

COMMITTENTE

Raoul Ragazzi

DATA

3 settembre 2026

OBIETTIVO

Trasferire il progetto senza perdere nulla: codice, dati, configurazioni, decisioni prese e problemi ancora aperti — per poi chiudere il lavoro.

INDICE

1. **Come usare questo documento** — 4 passi
2. **Regole della consegna** — come devono essere le risposte
3. **Sezione A** — Anagrafica, hosting e infrastruttura
4. **Sezione B** — Codice sorgente (i file veri)
5. **Sezione C** — Dati, bindings, segreti e servizi esterni
6. **Sezione D** — Cosa deve fare il sito (funzioni e flussi)
7. **Sezione E** — Problemi noti, buchi e regressioni
8. **Sezione F** — Contenuti, brand, SEO e legale
9. **Sezione G** — Definizione di "concluso"
10. **Sezione H** — Formato obbligatorio delle risposte
11. **Parte 2** — Cosa serve a me per pubblicare
12. **Parte 3** — Come procederò dopo aver ricevuto i file
13. **Allegato A** — Prompt pronto da incollare in ChatGPT
14. **Allegato B** — Checklist di consegna

1. Come usare questo documento

Questo non è un riassunto: è una richiesta di consegna. Serve a farsi restituire da ChatGPT tutto il materiale dei due siti in una forma che permetta di riprendere il lavoro da dove si è fermato, senza ricostruire nulla a memoria.

I 4 passi

1. **Apri una chat nuova con ChatGPT** per ciascun sito (una per vinoveritas, una per copilot.247agent.dev). Meglio la chat in cui hai sviluppato quel sito, così ha ancora il contesto: se non esiste più, va bene anche una chat nuova a cui allegghi i file che hai.
2. **Incolla il prompt dell'Allegato A** (ultima pagina) e allega questo PDF. Chiedi di rispondere *una sezione per volta*: A, poi B, poi C... Non tutto insieme, altrimenti taglia.
3. **Salva ogni risposta** in un file di testo o markdown. Un file per sezione va benissimo. Se ChatGPT spezza un file di codice in più parti, tienile in ordine e non modificarle.
4. **Passami il materiale**. Allegami i file in chat, oppure caricali nel repository GitHub: li leggo, ti dico file per file cosa va e cosa no, correggo e pubblichiamo `index.html` e `worker.js` nuovi.

REGOLA DI SICUREZZA — LEGGILA PRIMA DI TUTTO

In questo scambio non devono comparire **valori** di chiavi API, password, token o secret: solo i **nomi** delle variabili (es. `OPENAI_API_KEY`) e a cosa servono. Le chiavi si reinseriscono dopo, direttamente su Cloudflare, dove nessuna chat le vede.

Se in passato hai incollato chiavi vere dentro una chat o dentro `index.html / worker.js`, **rigenerale** (revoca la vecchia, crea la nuova) prima di andare online: è la prima cosa che verificherò nel codice che mi mandi.

Se ChatGPT dice che non ricorda

È normale e non è un problema: quello che conta davvero sono i **file attualmente online** (Sezione B) e l'elenco dei **problemi aperti** (Sezione E). Il resto si ricostruisce leggendo il codice. Dove non ricorda, deve scrivere `[DA VERIFICARE]` — mai inventare: un dettaglio inventato costa più tempo di un dettaglio mancante.

Se preferisci saltare ChatGPT

Puoi anche scaricare i file direttamente dalla dashboard Cloudflare (*Workers & Pages* → *il tuo Worker* → *Edit code*, oppure *Deployments* → *il deploy attivo*) e mandarmi quelli: sono la verità di cosa c'è online adesso. In quel caso rispondi tu, a voce tua, alle sole Sezioni E (problemi) e G (cosa vuol dire "finito"); a tutto il resto arrivo leggendo il codice.

2. Regole della consegna

Da comunicare a ChatGPT insieme al documento. Sono le regole che rendono il materiale utilizzabile invece che decorativo.

REGOLA	COSA SIGNIFICA IN PRATICA
File interi	Ogni file va incollato dalla prima all'ultima riga. Vietati i segnaposto tipo <code>// ...</code> resto invariato <code>...</code> , <code>/*</code> come prima <code>*/</code> , riassunti o estratti. Un file consegnato a metà è un file da riscrivere.
Nessun "miglioramento"	Il codice va consegnato <i>com'è online oggi</i> , bug compresi. Non va rifattorizzato, riordinato, rinominato o abbellito durante la consegna: alle correzioni ci penso io, ma devo partire dallo stato reale.
Una sezione per volta	Risposte lunghe vengono troncate. Si procede $A \rightarrow B \rightarrow C \rightarrow \dots$ aspettando un "vai" fra una sezione e l'altra.
File lunghi spezzati	Se un file non entra in un messaggio: <code>worker.js</code> – parte 1 di 3, parte 2 di 3 ... senza saltare righe e senza sovrapposizioni.
Intestazione di verifica	Prima di ogni file: percorso completo, numero totale di righe, testo della prima riga e dell'ultima. Così controllo che sia arrivato tutto.
Niente segreti	Solo i nomi delle variabili d'ambiente. Mai i valori.
Dubbi marcati	[DA VERIFICARE] su tutto ciò che non è certo, invece di colmare il vuoto con una supposizione plausibile.
Non applicabile	Se una domanda non riguarda il progetto: "non applicabile" e si prosegue. Nessuna domanda va saltata in silenzio.

3. Sezione A — Anagrafica, hosting e infrastruttura

Da compilare **separatamente per ciascuno dei due siti**.

RIF.	DOMANDA
A1	Identità. Nome del progetto, dominio di produzione esatto, eventuali domini di staging/anteprima, a cosa serve il sito in cinque righe, chi sono gli utenti.
A2	Piattaforma. Cloudflare Workers, Cloudflare Pages, o altro? Nome esatto del Worker/Progetto, ID account, zona/dominio collegato, ambienti configurati (production, preview, staging).
A3	Deploy attuale. Come va online una modifica, oggi, passo per passo: si incolla il codice nell'editor della dashboard Cloudflare? Si usa <code>wrangler deploy</code> ? C'è una build automatica da GitHub? Chi preme il bottone e da dove.
A4	Repository. Esiste un repo Git? URL, branch di produzione, chi ha accesso. Se non esiste: dove si trova la copia "buona" e aggiornata dei file (cartella locale, Drive, allegati in chat, solo dashboard Cloudflare?).
A5	Domini e DNS. Route e custom domain configurati sul Worker; chi gestisce il DNS del dominio (Cloudflare stesso? un altro registrar?); redirect apex/www; certificato HTTPS; eventuali sottodomini.
A6	Albero dei file. Elenco completo di tutti i file del progetto con percorso e ruolo di ciascuno: <code>index.html</code> , <code>worker.js</code> , CSS, JS aggiuntivi, <code>wrangler.toml</code> / <code>wrangler.jsonc</code> , <code>package.json</code> , <code>_headers</code> , <code>_redirects</code> , <code>robots.txt</code> , <code>sitemap.xml</code> , favicon, immagini, font.
A7	Runtime e vincoli. <code>compatibility_date</code> e <code>compatibility_flags</code> (è attivo <code>nodejs_compat</code> ?), librerie esterne usate e da dove arrivano (CDN? bundle? import da npm?), limiti già incontrati (tempo CPU, dimensione del bundle, sub-request).
A8	Relazione fra i due siti. <code>vinoveritas</code> e <code>copilot.247agent.dev</code> condividono qualcosa — codice, database, API, account, utenti? Uno chiama l'altro?

4. Sezione B — Codice sorgente

È la sezione più importante insieme alla E. Qui serve il codice, non la sua descrizione.

RIF.	DOMANDA
B1	<code>index.html</code> integrale — la versione realmente online adesso, dall'apertura <code><!doctype html></code> alla chiusura <code></html></code> , CSS e JavaScript inline compresi.
B2	<code>worker.js</code> integrale — idem: import iniziali, <code>export default { fetch... }</code> o <code>addEventListener</code> , tutte le funzioni di supporto, fino all'ultima riga.
B3	Tutti gli altri file elencati in A6, uno per uno, integrali: CSS, JS, HTML secondari, file di configurazione, JSON di dati.

B4	Corrispondenza con la produzione. Per ciascun file: la versione consegnata è esattamente quella online? Se non lo è (o non è certo), dirlo esplicitamente e indicare in cosa potrebbe differire.
B5	Codice non attivo. Funzioni scritte e mai collegate, blocchi commentati che vanno tenuti, TODO / FIXME , versioni alternative rimaste nel file: segnalarli, così non li tratto come spazzatura né come codice vivo.
B6	Cronologia. Le ultime 5-10 modifiche fatte al progetto, in ordine, con cosa dovevano ottenere e se hanno funzionato.

Intestazione richiesta prima di ogni file:

```
FILE: worker.js
RIGHE: 412
PRIMA RIGA:  export default {
ULTIMA RIGA: };
IN PRODUZIONE: sì / no / [DA VERIFICARE]
-----
<contenuto integrale>
```

5. Sezione C — Dati, bindings, segreti e servizi esterni

RIF.	DOMANDA
C1	Bindings Cloudflare in uso. Per ognuno: tipo, nome della risorsa, nome del binding usato nel codice (<code>env.QUALCOSA</code>) e a cosa serve. Da coprire: <i>KV</i> (namespace, formato delle chiavi, esempio di valore), <i>D1</i> , <i>R2</i> (bucket, tipo di file), <i>Durable Objects</i> , <i>Queues</i> , <i>Workers AI</i> , <i>Vectorize</i> , <i>Email/send_email</i> , <i>Service bindings</i> , <i>Cron Triggers</i> .
C2	Database. DDL completo: <code>CREATE TABLE</code> e <code>CREATE INDEX</code> di ogni tabella, con tipi e vincoli. Più 2-3 righe di esempio anonimizzate per tabella, per capire il formato reale dei dati.
C3	Variabili d'ambiente e secret. Tabella: <i>nome</i> · <i>a cosa serve</i> · <i>dove si ottiene il valore</i> · <i>è già configurato su Cloudflare?</i> — senza i valori .
C4	Servizi di terze parti. Per ciascuno (LLM, pagamenti, email, analytics, CRM, captcha, mappe, webhook...): quale servizio, quale endpoint viene chiamato, con che metodo, che corpo si manda, che risposta si aspetta, come sono gestiti errori e limiti di frequenza, su quale piano/costo.
C5	Dati di produzione da preservare. Ci sono già utenti, ordini, contenuti, iscrizioni? Quanti record, in quali tabelle/namespaces, esiste un export o un backup? Si possono perdere o no?
C6	Autenticazione tecnica. Come il Worker si autentica verso i servizi esterni (header, firma, OAuth), e come verifica le chiamate in entrata (token, CORS, origini ammesse, rate limit).

6. Sezione D — Cosa deve fare il sito

Il codice dice cosa *fa*. Questa sezione dice cosa *dovrebbe* fare: è la differenza fra le due che genera i "buchi".

RIF.	DOMANDA
D1	Elenco funzionalità numerato. Per ciascuna: nome, cosa fa, chi la usa, e stato onesto — <i>funziona</i> / <i>funziona a metà</i> / <i>rotta</i> / <i>mai iniziata</i> .
D2	Flussi utente passo per passo, dal primo clic al risultato finale, per ogni percorso principale (es. visita → form → conferma → email; oppure login → area riservata → azione).
D3	Rotte del Worker. Per ognuna: metodo, percorso, parametri accettati, esempio di richiesta, esempio di risposta corretta, esempi di risposta d'errore, chi ha il diritto di chiamarla.
D4	Accessi e ruoli. Esiste un login? Dove vivono sessioni/cookie/token, quanto durano, quali ruoli esistono e chi vede cosa. Esiste un'area di amministrazione?
D5	Regole non scritte. Prezzi, limiti, testi obbligatori, condizioni, comportamenti dati per scontati nelle conversazioni ma mai finiti nel codice o nella documentazione.

D6	Requisiti trasversali. Lingue supportate, comportamento su mobile, browser da supportare, accessibilità, prestazioni attese, gestione degli stati di caricamento e di errore lato utente.
-----------	--

7. Sezione E — Problemi noti, buchi e regressioni

LA SEZIONE DECISIVA

È il motivo della migrazione. Va compilata con precisione clinica: senza giustificazioni, senza attenuazioni, senza "dovrebbe funzionare". Serve l'elenco di ciò che non va, come si riproduce e cosa è già stato tentato.

RIF.	DOMANDA
E1	Tabella dei bug. Una riga per problema, con le colonne: <i>sintomo osservato</i> · <i>pagina o funzione</i> · <i>passi esatti per riprodurlo</i> · <i>da quando</i> · <i>frequenza (sempre / a volte / una volta)</i> · <i>errore visibile in console o nei log</i> .
E2	Buchi. Cose richieste e mai completate: cosa era stato chiesto, cosa è stato consegnato al posto suo, cosa manca ancora.
E3	Regressioni. Funzionalità che prima funzionavano e si sono rotte: quale modifica le ha rotte e in che occasione.
E4	Tentativi già fatti. Per ogni bug importante: cosa è stato provato per risolverlo e perché non ha funzionato. Serve a non ripetere gli stessi tentativi.
E5	Zone fragili. Parti riscritte molte volte, punti di cui non ci si fida, codice che "si rompe se lo tocchi".
E6	Log ed errori reali. Copia-incolla degli errori dalla console del browser (F12 → Console e Network) e dai log di Cloudflare (<i>Workers & Pages</i> → <i>Worker</i> → <i>Logs</i> , oppure <code>wrangler tail</code>). Anche solo screenshot descritti a parole vanno bene.
E7	Differenze fra ambienti. Qualcosa funziona in locale/anteprima ma non in produzione (o viceversa)? Cosa, e da quando.

8. Sezione F — Contenuti, brand, SEO e legale

RIF.	DOMANDA
F1	Testi. I testi attualmente online sono definitivi o provvisori? Esiste una versione approvata altrove? In quali lingue.
F2	Immagini, logo, font. Quali file, dove sono ospitati (R2, CDN esterna, base64 dentro l'HTML?), con quali licenze d'uso.
F3	Identità visiva. Palette colori, caratteri, spaziature, riferimenti da rispettare; cosa non va toccato per nessun motivo.
F4	SEO. <title> e meta description per pagina, immagine Open Graph, robots.txt, sitemap.xml, dati strutturati, proprietà verificata su Google Search Console.
F5	Analytics e tracciamento. Cosa è installato (Google Analytics, Cloudflare Web Analytics, Meta Pixel...), con quale ID, e se ci sono conversioni o eventi configurati.
F6	Legale. Privacy policy, cookie banner, consenso GDPR, dati personali raccolti dai form, dove vengono salvati e per quanto tempo, chi è il titolare del trattamento.

9. Sezione G — Definizione di "concluso"

Questa sezione la scrive Raoul, non ChatGPT: è il criterio con cui misureremo la fine del lavoro.

RIF.	DOMANDA
G1	Condizioni di completamento. Per ciascun sito, l'elenco delle cose che devono essere vere perché il sito si possa dichiarare finito. Frasi verificabili: "il form di contatto invia l'email e mostra la conferma", non "il sito funziona bene".
G2	Priorità. Cosa serve subito, cosa può attendere, cosa si può tagliare se costa troppo.
G3	Scadenze. Date, impegni presi con clienti o terzi, eventi legati alla pubblicazione.
G4	Uso reale. I siti sono già usati da qualcuno adesso? Se sì, quali interruzioni sono tollerabili durante la migrazione e in quali fasce orarie.
G5	Budget tecnico. Vincoli di costo sui servizi esterni (piano Cloudflare, crediti API, limiti mensili) da rispettare nelle soluzioni che proporrò.

10. Sezione H — Formato obbligatorio delle risposte

Istruzioni operative per ChatGPT. Riepilogano le regole del capitolo 2.

1. Rispondi **una sezione per volta**, in ordine (A, B, C, D, E, F), e fermati aspettando "vai".
2. I file di codice si consegnano **integrali**, con l'intestazione di verifica (percorso, righe totali, prima e ultima riga).
3. Nessun segnaposto, nessun riassunto, nessuna omissione: se non entra in un messaggio, si spezza in parti numerate.
4. Non riscrivere, non riordinare, non correggere il codice mentre lo consegna.
5. Marca [DA VERIFICARE] tutto ciò che non è certo. Non colmare i vuoti con ipotesi.
6. Mai valori di chiavi, token o password: solo i nomi delle variabili.
7. Se una domanda non si applica: "non applicabile", e avanti. Nessuna domanda saltata in silenzio.
8. Se possibile, alla fine raccogli tutto in **un unico file** `.md` scaricabile, con una intestazione per sezione.

11. Parte 2 — Cosa serve a me per pubblicare

Questa parte riguarda te e me, non ChatGPT. Sono le condizioni pratiche perché io possa non solo correggere i file, ma anche metterli online.

Per lavorare sul codice — basta uno di questi

- **Repository GitHub** (consigliato): carichi lì i file dei due siti e io lavoro direttamente sul repo, con la cronologia delle modifiche. Ogni versione resta recuperabile: se una modifica peggiora le cose, si torna indietro in un istante.
- **Allegati in chat**: mi passi i file qui, li leggo, ti restituisco le versioni corrette da incollare tu. Funziona, ma senza cronologia.

Per pubblicare su Cloudflare — scegli tu

MODALITÀ	COSA SERVE	NOTA
Pubblico io	Un <i>API token</i> Cloudflare con i permessi: <code>Workers Scripts: Edit</code> , <code>Workers KV Storage: Edit</code> , <code>D1: Edit</code> , <code>Account Settings: Read</code> , più <code>Zone: DNS: Edit</code> solo se dobbiamo toccare i domini. Serve anche l' <i>Account ID</i> .	Più veloce e verificabile: pubblico e controllo subito il risultato dal vivo.
Pubblichi tu	Nessun accesso. Ti consegno <code>index.html</code> e <code>worker.js</code> finiti e le istruzioni esatte, tu li incolli nella dashboard.	Nessuna credenziale in giro, ma ogni giro di prova passa da te.

SE SCEGLI DI DARMI IL TOKEN

Crea un token **nuovo e dedicato** a questo lavoro (non riusare uno esistente), con i soli permessi elencati e, se possibile, una scadenza. A lavoro finito lo revochi da *My Profile* → *API Tokens*. Non passarlo dentro un file che finisce nel repository.

Informazioni minime che mi servono comunque

- ☐ I due domini esatti e quale Worker serve quale dominio.
- ☐ I file attualmente online (Sezione B) — anche presi dalla dashboard Cloudflare.
- ☐ L'elenco dei problemi (Sezione E), anche solo a voce tua, senza formalità.
- ☐ Cosa significa "finito" per te (Sezione G1), sito per sito.
- ☐ Se ci sono dati di produzione da non perdere (Sezione C5).

12. Parte 3 — Come procederò

1. **Audit.** Leggo tutto il materiale e ti do un referto chiaro, file per file: cosa è sano, cosa è rotto, cosa è pericoloso (chiavi esposte, dati non protetti), cosa manca del tutto. Senza addolcire il giudizio.
2. **Piano.** Elenco di interventi ordinati per priorità, con una stima di quanto pesa ciascuno. Lo approvi tu prima che io tocchi qualcosa.
3. **Correzione.** Riscrivo o riparo `index.html` e `worker.js` (e gli altri file), un intervento per volta, spiegando cosa cambia e perché.
4. **Verifica.** Provo davvero: rotte, form, integrazioni esterne, comportamento su mobile, casi d'errore. Se qualcosa non funziona te lo dico, non lo nascondo.
5. **Pubblicazione.** Andiamo online e controlliamo il sito dal vivo, con la possibilità di tornare indietro se qualcosa non va.
6. **Documentazione.** Lascio nel repository un `README` che spiega com'è fatto il sito, come si pubblica e dove si mettono le mani. Così il progetto non dipende più dalla memoria di una chat — né della mia.

13. Allegato A — Prompt da incollare in ChatGPT

Copia il testo qui sotto, sostituisci il nome del sito, allega questo PDF e invia.

Sto trasferendo lo sviluppo di questo progetto a un altro assistente.
Ti allego il documento di consegna: seguilo alla lettera.

PROGETTO: <vinoveritas / copilot.247agent.dev>

Il tuo compito è consegnare il progetto, non migliorarlo. Regole:

1. Rispondi una sezione per volta, nell'ordine A, B, C, D, E, F.
Alla fine di ogni sezione fermati e aspetta che io scriva "vai".
2. I file di codice vanno consegnati INTEGRALI, dalla prima all'ultima riga, nella versione che è online adesso. Sono vietati i segnaposto ("... resto invariato ...", "come sopra", riassunti, estratti).
Se un file non entra in un messaggio, spezzalo in parti numerate ("worker.js - parte 1 di 3") senza saltare nulla.
3. Prima di ogni file scrivi questa intestazione:
FILE: <percorso>
RIGHE: <numero totale>
PRIMA RIGA: <testo>
ULTIMA RIGA: <testo>
IN PRODUZIONE: sì / no / [DA VERIFICARE]
4. NON riscrivere, non riordinare, non correggere e non abbellire il codice mentre lo consegni. Mi serve com'è oggi, bug inclusi.
5. Scrivi [DA VERIFICARE] su tutto ciò che non ricordi con certezza.
Non inventare mai un dettaglio per riempire un vuoto: un dato mancante costa poco, un dato inventato costa moltissimo.
6. Non scrivere MAI valori di chiavi API, token, password o secret:
solo i NOMI delle variabili e a cosa servono.
7. Se una domanda non si applica, scrivi "non applicabile" e prosegui.
Non saltare nessuna domanda in silenzio.
8. Nella sezione E (problemi noti) sii spietatamente onesto: elenca tutto ciò che non funziona, ciò che ti avevo chiesto e non è mai stato completato, e cosa hai già provato senza successo. Questa è la sezione più importante di tutte.

Comincia dalla Sezione A del documento allegato.

VARIANTE BREVE — SE NON HAI IL PDF SOTTOMANO

«Consegnami il progetto <nome> per passarlo a un altro sviluppatore. Una sezione per volta: **(A)** hosting, dominio e come si pubblica oggi; **(B)** tutti i file di codice integrali, com'è online adesso, senza omissioni e senza migliorie; **(C)** database, bindings e servizi esterni — solo i nomi delle chiavi, mai i valori; **(D)** cosa deve fare il sito, funzione per funzione, con lo stato reale di ciascuna; **(E)** tutti i bug noti con i passi per riprodurli e cosa hai già provato; **(F)** contenuti, SEO e aspetti legali. Marca [DA VERIFICARE] ciò di cui non sei certo.»

14. Allegato B — Checklist di consegna

Da spuntare per **ciascuno** dei due siti. Quando le voci in grassetto sono tutte spuntate, posso già cominciare: il resto si può recuperare strada facendo.

CODICE

- ☐ `index.html` **ricevuto integrale** (verificata l'ultima riga)
- ☐ `worker.js` **ricevuto integrale** (verificata l'ultima riga)
- ☐ Tutti gli altri file elencati e ricevuti
- ☐ `wrangler.toml` / `wrangler.jsonc` ricevuto
- ☐ Confermato che i file corrispondono a ciò che è online

INFRASTRUTTURA

- ☐ **Dominio e nome del Worker confermati**
- ☐ Modalità di deploy attuale descritta
- ☐ Bindings elencati (KV, D1, R2, AI, Email...)
- ☐ Schema del database ricevuto
- ☐ Nomi delle variabili d'ambiente ricevuti — **senza valori**

COMPORTAMENTO

- ☐ Elenco delle funzionalità con lo stato reale di ciascuna
- ☐ Rotte del Worker documentate
- ☐ **Elenco dei bug con i passi per riprodurli**
- ☐ Tentativi di correzione già fatti
- ☐ Errori reali da console e log

DECISIONI TUE

- ☐ **Definizione di "concluso" scritta, sito per sito**
- ☐ Priorità stabilite
- ☐ Scelto chi pubblica (io con token / tu dalla dashboard)
- ☐ Chiavi eventualmente esposte nelle vecchie chat: **rigenerate**

IN SINTESI

Se ottieni solo due cose da ChatGPT, che siano **i file integrali** (Sezione B) e **l'elenco onesto di ciò che non funziona** (Sezione E). Con quelle due parto; tutto il resto lo ricostruisco leggendo il codice e chiedendo a te.